

Basic (typed function + custom error) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/custom_errors_practice.py`, under its `# Basic` section — the same file also holds the [Failure handling exercise](#), under a `# Failure handling` section. Run it with `cd practice && python custom_errors_practice.py`.

Used later by: the [Real-world wiring exercise](#) and the [Build Task](#). Neither imports this file — they re-write the same one-line custom-error pattern as `MissingConfigError` in `practice/config_logging_wiring/exceptions.py` and `practice/build_task/exceptions.py`.

Basic Version

Approach 1 — the direct way

```
# practice/custom_errors_practice.py – Basic section
class InvalidAgeError(Exception):
    pass

def set_age(age):
    if age < 0:
        raise InvalidAgeError("age cannot be negative: " + str(age))

# test it
set_age(25)  # this works fine, nothing happens

try:
    set_age(-5)
except InvalidAgeError as e:
    print("Caught it:", e)
```

Expected output:

```
Caught it: age cannot be negative: -5
```

This version works correctly. It's missing type hints, and it builds the message with `+` instead of an f-string — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — f-string message

```
# practice/custom_errors_practice.py – Basic section
class InvalidAgeError(Exception):
    pass

def set_age(age: int) -> None:
    if age < 0:
        raise InvalidAgeError(f"age cannot be negative: {age}")

set_age(25) # runs fine, no output

try:
    set_age(-5)
except InvalidAgeError as e:
    print(f"Caught it: {e}")
```

Expected output:

```
Caught it: age cannot be negative: -5
```

Approach 2 — a guard clause with an early return check, and a docstring

```
# practice/custom_errors_practice.py – Basic section
class InvalidAgeError(Exception):
    """Raised when an age value is negative."""
    pass

def set_age(age: int) -> None:
    """Validate an age. Raises InvalidAgeError if age is negative."""
    if age >= 0:
        return
    raise InvalidAgeError(f"age cannot be negative: {age}")
```

```
set_age(25)

try:
    set_age(-5)
except InvalidAgeError as e:
    print(f"Caught it: {e}")
```

Expected output:

```
Caught it: age cannot be negative: -5
```

Difference from Basic: both Intermediate approaches add full type hints — `age: int , -> None` — turning the function's signature into documentation a reader (or an editor's autocomplete) can rely on without opening the function body. Approach 2 also adds docstrings and flips the condition into an early-return guard clause, which some style guides prefer because it keeps the "normal path" unindented and the error path clearly separate — purely a style choice, not a behavior difference from Approach 1.

Which one should you actually write? Intermediate Approach 1 (f-string, type hints, no extra machinery) is completely correct and what most people should default to.